

SLURM

How to Submit Jobs.

Aaron Nugent

Version 1.0.1



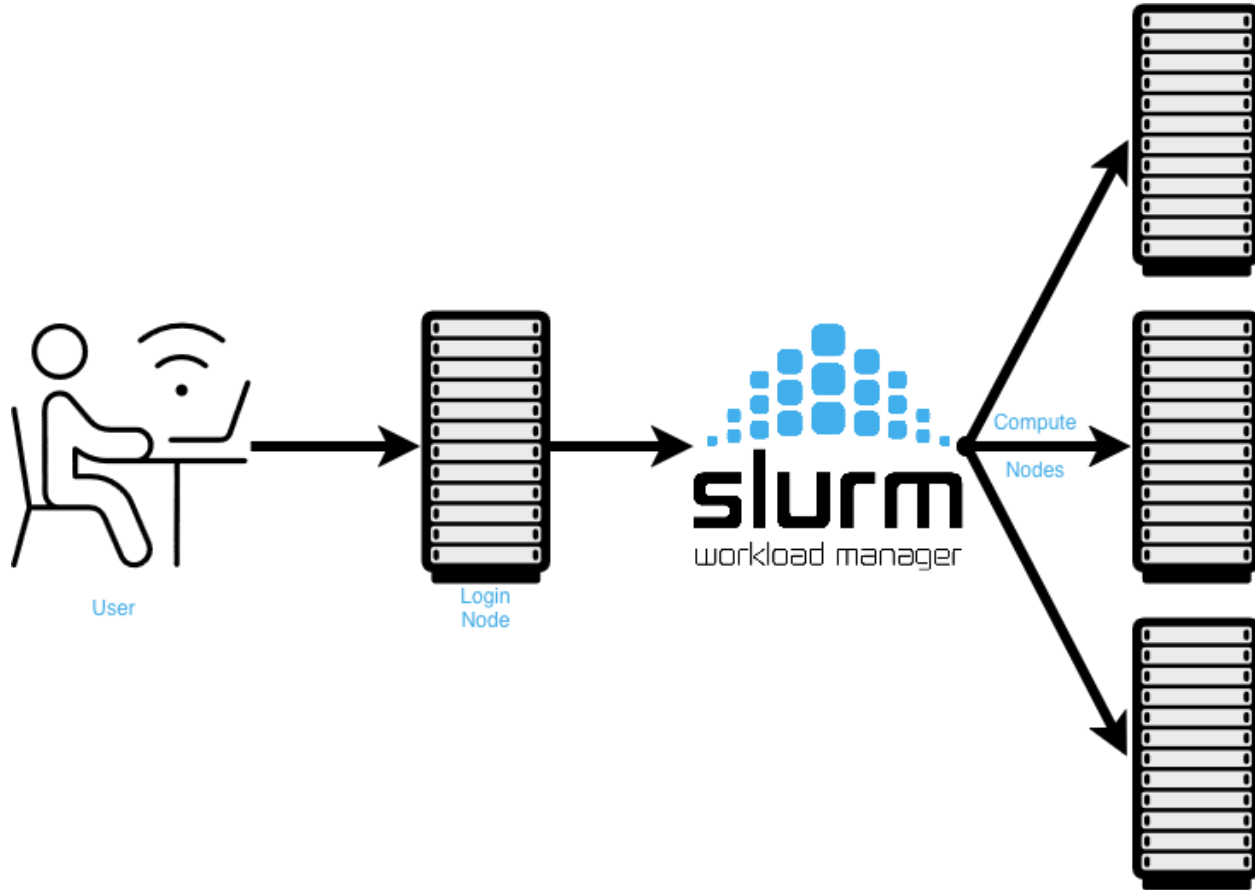
What is SLURM?

SLURM stands for **S**imple **L**inux **U**tility for **R**esource **M**anagement.

SLURM is an open source job scheduling system for Linux clusters, it has three key functions:

- It allocates access to resources (compute nodes) to users.
- It starts, executes and monitors jobs.
- It manages the queue of pending work.

SLURM in HPC



Meluxina computational resources are controlled by SLURM. User jobs are submitted to a queue where they are held until resources become available. A job and its requirements can be submitted in one of two ways;

Meluxina computational resources are controlled by SLURM. User jobs are submitted to a queue where they are held until resources become available. A job and its requirements can be submitted in one of two ways;

Batch Mode

Users submit a script **launcher** file to SLURM, the commands and applications inside are run by SLURM.

Meluxina computational resources are controlled by SLURM. User jobs are submitted to a queue where they are held until resources become available. A job and its requirements can be submitted in one of two ways;

Batch Mode

Users submit a script **launcher** file to SLURM, the commands and applications inside are run by SLURM.

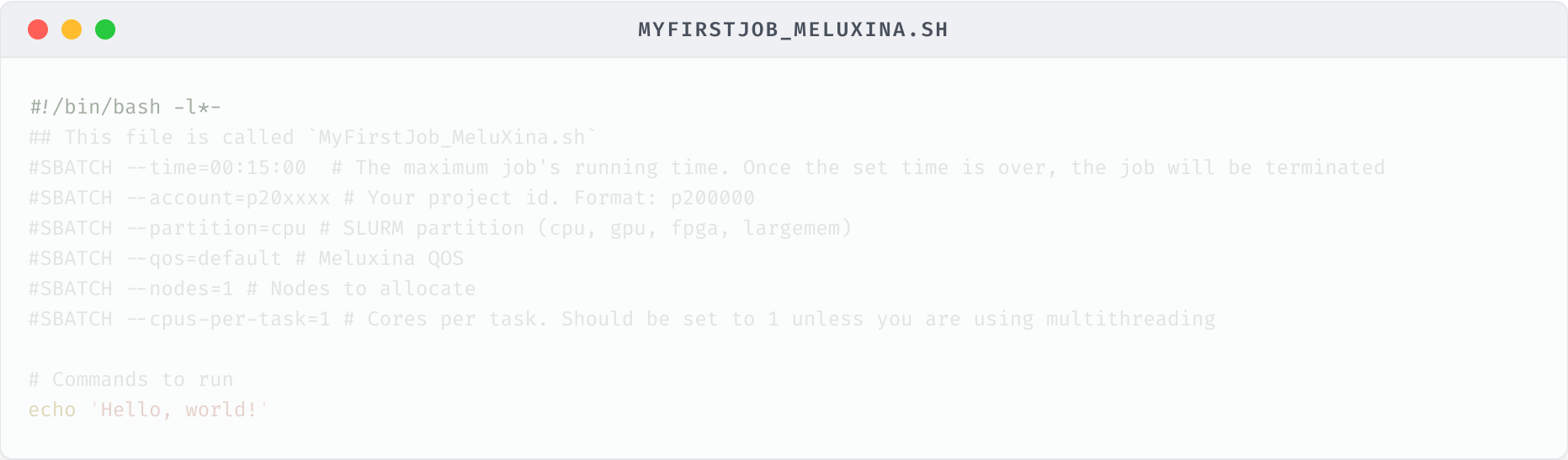
Interactive Mode

Users get connected by SLURM to a (set of) computing nodes directly and can run their applications **interactively**.

Batch File Structure

When running jobs on MeluXina using a batch file, the following parameters are important: a section specific for instructing SLURM, and a section for user commands. At the top of the batch file a set of commands with the prefix '**#SBATCH**' tell SLURM the resources desired and the users details.

Below that is where the user can write their commands to run their job.



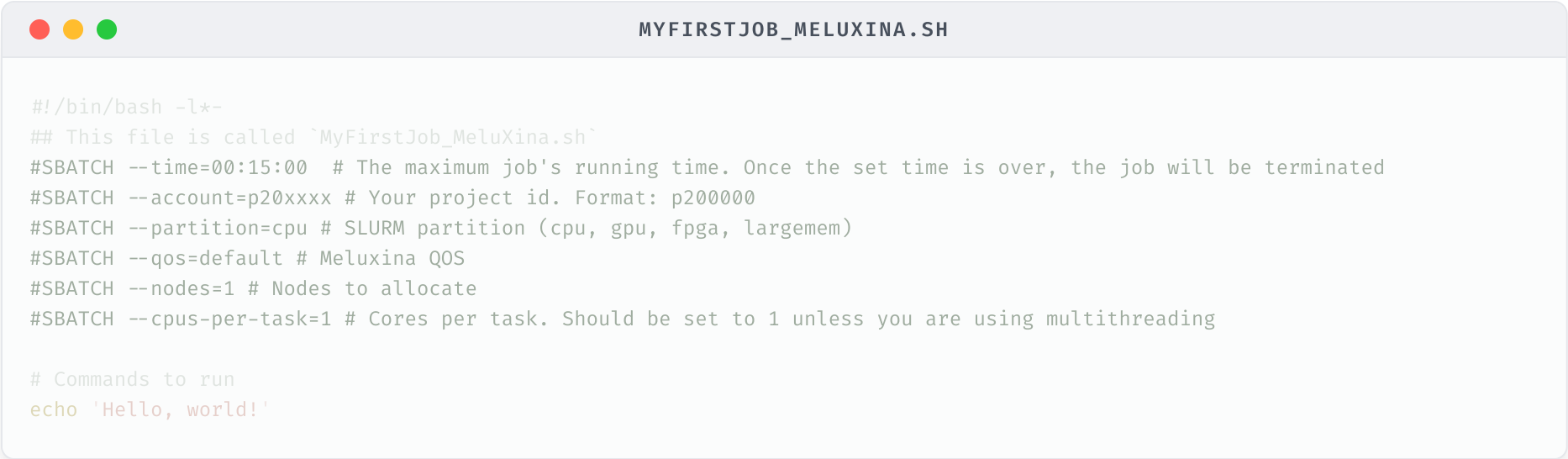
```
#!/bin/bash -l*-
## This file is called `MyFirstJob_MeluXina.sh`
#SBATCH --time=00:15:00 # The maximum job's running time. Once the set time is over, the job will be terminated
#SBATCH --account=p20xxxx # Your project id. Format: p200000
#SBATCH --partition=cpu # SLURM partition (cpu, gpu, fpga, largemem)
#SBATCH --qos=default # Meluxina QOS
#SBATCH --nodes=1 # Nodes to allocate
#SBATCH --cpus-per-task=1 # Cores per task. Should be set to 1 unless you are using multithreading

# Commands to run
echo 'Hello, world!'
```

Batch File Structure

When running jobs on MeluXina using a batch file, the following parameters are important: a section specific for instructing SLURM, and a section for user commands. At the top of the batch file a set of commands with the prefix '**#SBATCH**' tell SLURM the resources desired and the users details.

Below that is where the user can write their commands to run their job.



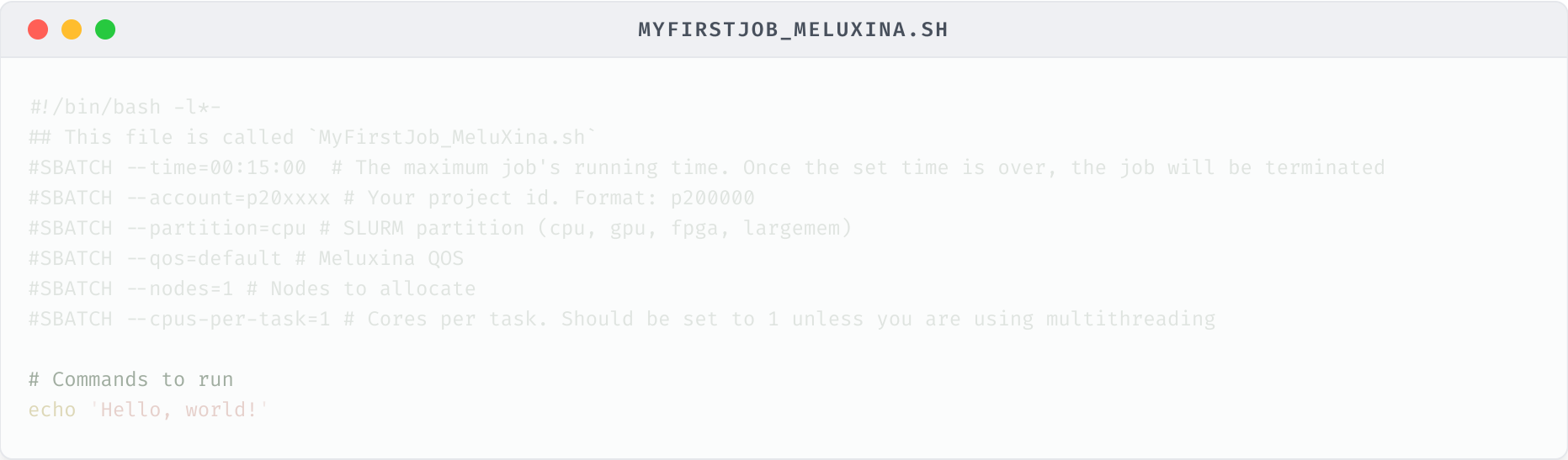
```
#!/bin/bash -l*-
## This file is called `MyFirstJob_MeluXina.sh`
#SBATCH --time=00:15:00 # The maximum job's running time. Once the set time is over, the job will be terminated
#SBATCH --account=p20xxxx # Your project id. Format: p200000
#SBATCH --partition=cpu # SLURM partition (cpu, gpu, fpga, largemem)
#SBATCH --qos=default # Meluxina QOS
#SBATCH --nodes=1 # Nodes to allocate
#SBATCH --cpus-per-task=1 # Cores per task. Should be set to 1 unless you are using multithreading

# Commands to run
echo 'Hello, world!'
```

Batch File Structure

When running jobs on MeluXina using a batch file, the following parameters are important: a section specific for instructing SLURM, and a section for user commands. At the top of the batch file a set of commands with the prefix '**#SBATCH**' tell SLURM the resources desired and the users details.

Below that is where the user can write their commands to run their job.



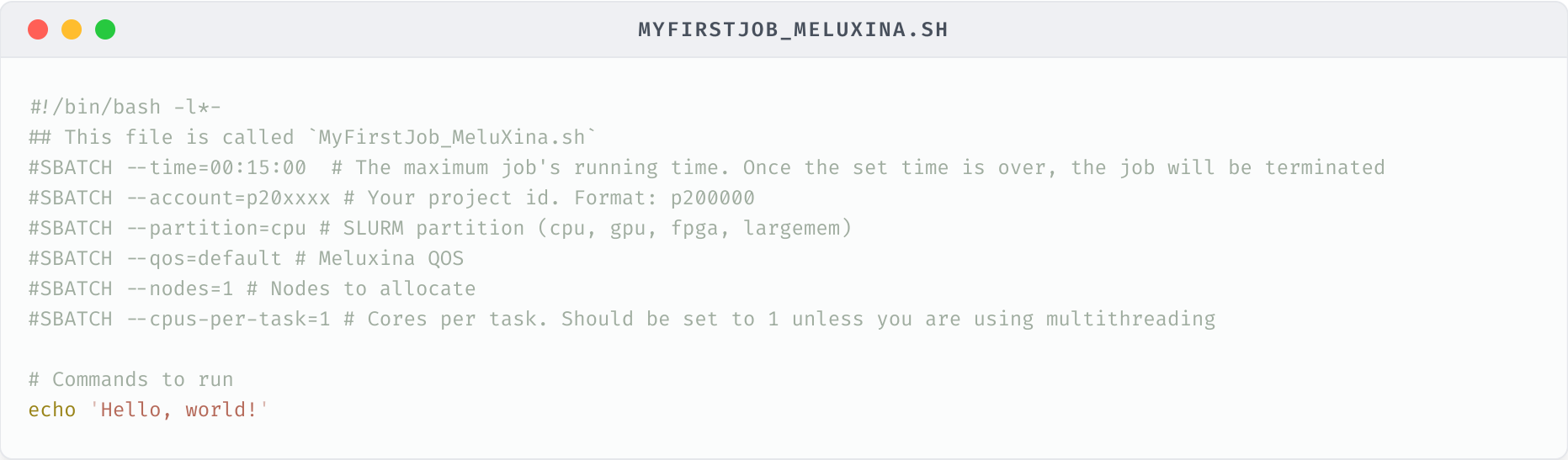
```
#!/bin/bash -l*-
## This file is called `MyFirstJob_MeluXina.sh`
#SBATCH --time=00:15:00 # The maximum job's running time. Once the set time is over, the job will be terminated
#SBATCH --account=p20xxxx # Your project id. Format: p200000
#SBATCH --partition=cpu # SLURM partition (cpu, gpu, fpga, largemem)
#SBATCH --qos=default # Meluxina QOS
#SBATCH --nodes=1 # Nodes to allocate
#SBATCH --cpus-per-task=1 # Cores per task. Should be set to 1 unless you are using multithreading

# Commands to run
echo 'Hello, world!'
```

Batch File Structure

When running jobs on MeluXina using a batch file, the following parameters are important: a section specific for instructing SLURM, and a section for user commands. At the top of the batch file a set of commands with the prefix '**#SBATCH**' tell SLURM the resources desired and the users details.

Below that is where the user can write their commands to run their job.



```
#!/bin/bash -l*-
## This file is called `MyFirstJob_MeluXina.sh`
#SBATCH --time=00:15:00 # The maximum job's running time. Once the set time is over, the job will be terminated
#SBATCH --account=p20xxxx # Your project id. Format: p200000
#SBATCH --partition=cpu # SLURM partition (cpu, gpu, fpga, largemem)
#SBATCH --qos=default # Meluxina QOS
#SBATCH --nodes=1 # Nodes to allocate
#SBATCH --cpus-per-task=1 # Cores per task. Should be set to 1 unless you are using multithreading

# Commands to run
echo 'Hello, world!'
```

SLURM Partitions

Note

By default, SLURM will redirect both the standard output (stdout) and error (stderr) streams for your job to a file named slurm-JOBNUMBER.out in the directory where you submitted the SLURM script.

Partition	Default Time	Max. Time	Description
cpu	no default, users must specifytime limit	set by QOS	Default partition, MeluXina Cluster Module
gpu	no default, users must specifytime limit	set by QOS	MeluXina Accelerator Module - GPU Nodes
fpga	no default, users must specifytime limit	set by QOS	MeluXina Accelerator Module - FPGA Nodes
largemem	no default, users must specifytime limit	set by QOS	MeluXina Large Memory Module

How to Run an Interactive Job

A compute node can also be accessed directly from the log in node allowing the user to run jobs interactively.

Compute nodes are reserved with the command `salloc`.

Example

```
salloc -A YOURPROJECT --reservation=cpudev -q dev -N 1 -t 2:0:0
```

From here scripts can be run as normal using Python, srun, etc.

Modules can be loaded directly in the terminal.

Useful Commands

Command	Description
sbatch	Submit a batch script to Slurm for processing.
squeue --me	Show information about jobs in the queue. The <code>--me</code> flag shows only your job(s) in the queue.
scancel	End or cancel a queued job.
sinfo	Get information about the resources on available nodes that make up the HPC cluster.
salloc	Start an interactive job.
sacct	Check the job accounting

Practical: Serial Meluxina

- Create a file named ***hello_world_serial.cpp*** and add the following to it:

```
#include <iostream>
int main()
{
    std::cout << "Hello, World from thread 0 out of 1 from process 0 out of 1\n";
}
```

- Create a new file named ***serial.sh*** and enter your slurm batch file information.
- At the bottom of the batch file include the following:

```
#module load GCC
g++ hello_world_serial.cpp -o hello_world_serial
sleep 30
srun ./hello_world_serial
```

- Submit the job.
- View the output file.
- Check the job afterwards using the `sacct` command